Studio Nx

JS 133 — waveformColumns()

2026-10-14

JS 133 — `waveformColumns()` (오디오 클립 파형 접기)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 열아홉 번째 함수 (FR-210 오디오 클립 파형)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	파형을 캔버스에 그리는 함수(그리기)를 이어서 진행합니다

1. 하는 일

오디오 클립의 파형을 **클립 폭(칸)에 맞게 접어** 한 칸마다 진폭을 돌려줍니다.

- `peaks`(원본 소스 전체 진폭)를 `sourceLengthTick`에 걸쳐 있다고 봅니다.
- 클립은 소스의 `offsetTick`부터 `lengthTick`만큼만 보여 줍니다(트리밍 반영).
- 그 구간을 가로 `columns`칸으로 나누고, **한 칸이 덮는 구간의 가장 큰 진폭**(절댓값)을 그 칸 값으로 씁니다.

```
peaks  [0.1, 0.5, 0.9, 0.3]   소스 4틱, offset 0, 길이 4
4칸  → [0.1, 0.5, 0.9, 0.3]     (그대로)
2칸  → [0.5, 0.9]               (큰 값만)
8칸  → [0.1,0.1, 0.5,0.5, 0.9,0.9, 0.3,0.3]
```

peaks	offset	길이	소스	칸	결과
[0.1,0.5,0.9,0.3]	0	4	4	2	[0.5, 0.9]
[0.1,0.5,0.9,0.3]	1	2	4	2	[0.5, 0.9]
[0.1,0.5,0.9,0.3]	2	2	4	1	[0.9]
[-0.7, 0.2]	0	2	2	2	[0.7, 0.2]
[]	0	4	4	3	[0, 0, 0]

2. 코드

```
export function waveformColumns(peaks, offsetTick, lengthTick, sourceLengthTick, columns) {
  if (!Array.isArray(peaks)) throw new RangeError('peaks 는 배열이어야 합니다');
  if (!Number.isInteger(columns) || columns < 1) throw new RangeError('columns 는 1 이상의 정수여야 합니
다: ' + columns);
  if (typeof sourceLengthTick !== 'number' || !isFinite(sourceLengthTick) || sourceLengthTick <= 0) thro
w new RangeError('sourceLengthTick 는 0 보다 큰 숫자여야 합니다: ' + sourceLengthTick);
  const out = new Array(columns).fill(0);
  const n = peaks.length;
  if (n === 0) return out;
  const peaksPerTick = n / sourceLengthTick;
  for (let c = 0; c < columns; c++) {
    const t0 = (offsetTick + (c / columns) * lengthTick) * peaksPerTick;
    const t1 = (offsetTick + ((c + 1) / columns) * lengthTick) * peaksPerTick;
    let i0 = Math.floor(t0);
    let i1 = Math.ceil(t1);
```

```
    if (i1 <= i0) i1 = i0 + 1;
    i0 = Math.max(0, Math.min(n - 1, i0));
    i1 = Math.max(i0 + 1, Math.min(n, i1));
    let max = 0;
    for (let i = i0; i < i1; i++) {
      const v = Math.abs(peaks[i]);
      if (v > max) max = v;
    }
    out[c] = max;
  }
  return out;
}
```

최신 문법으로 쓰면 (공부용)

```
export function waveformColumns(peaks, offsetTick, lengthTick, sourceLengthTick, columns) {
  const inRange = peaks.slice(); // 그대로
  return Array.from({ length: columns }, (_, c) => {
    const at = (k) => (offsetTick + (k / columns) * lengthTick) * (peaks.length / sourceLengthTick);
    const i0 = Math.max(0, Math.floor(at(c)));
    const i1 = Math.max(i0 + 1, Math.min(peaks.length, Math.ceil(at(c + 1))));
    return Math.max(0, ...peaks.slice(i0, i1).map(Math.abs));
  });
}
```

최신 문법	실제 코드	뜻
`Array.from({length}, ...)`	`for` + `out[c]`	같은 결과입니다. 칸마다 계산을 눈으로 따라가기 쉽게 반복으로 씁니다
`Math.max(0, ...배열)`	`for` + 비교	큰 배열에서는 스프레드가 인자 한계에 걸립니다. 반복이 안전합니다
화살표 함수 `at`	식을 그대로 폄	같은 식입니다

3. 왜 이렇게

선택	이유
소스 좌표 → 칸 좌표 두 번 나눔	`tick → peak` 인덱스로 한 번, `칸 → tick`으로 한 번. 트리밍(offset)이 있든 없든 같은 식으로 처리됩니다
한 칸 = 구간의 최댓값	파형을 줄일 때 큰 소리가 사라지면 안 됩니다. 대표값으로 최댓값을 씁니다
절댓값(`Math.abs`)	파형은 위아래 어느 쪽이든 크기가 중요합니다. 부호는 그릴 때 위/아래로 다시 나눕니다
칸보다 peak가 적으면 이웃으로 채움	`i1 <= i0`이면 한 칸이 peak 하나보다 좁습니다. 가장 가까운 peak 하나를 씁니다
빈 peaks는 0으로	파형이 없어도 클립 폭만큼 자리를 잡아 둡니다(레이아웃 흔들림 방지)

4. 시험 (Node에서 직접 실행, 13개 통과)

확인	결과
칸수=peak수(그대로) · 절반(큰 값) · 두 배(각 peak 두 칸)	✓
offset·길이로 일부만(2칸·1칸) · 소스가 클립보다 길 때	✓
음수는 절댓값으로	✓

확인	결과
빈 peaks → 0으로 채움	✓
결과 길이 = columns	✓
peaks 배열 아님·칸 0·칸 소수·소스길이 0 → 오류	✓